ChatGPT

Dynamic MicroFlowGroup Window-Capacity Design for Pastey

Executive summary

Pastey's current scheduler already thinks in **integer requested-window budget**, and the ordinary file path is already **batch-relative and size-weighted**. The part that is now out of alignment is MicroFlowGroup: ordinary transfers are budgeted in "window share" terms, but MicroFlowGroup admission is still defined by fixed byte cliffs such as child size and group size thresholds. The design docs also make clear that MicroFlowGroup is currently a **scheduler-only, serial child-dispatch abstraction**, not a protocol object, not a bundle/archive, and not a binary-v2 transport.

The strongest recommendation is therefore **not** to make MicroFlowGroup "throughput-adaptive" right away, and **not** to simply raise fixed MiB thresholds. The right next step is to replace fixed admission cutoffs with a **dynamic one-window service quantum** and to keep that model **planner-side, batch-static, and contention-aware**: a group should only exist when it relieves actual planner pressure, and only if its aggregate service cost fits within what one planner window should reasonably carry in the current batch. That preserves the current 7+1 style behavior for huge-plus-many-small workloads, while avoiding needless serialization when the queue is uncongested. This matches the current project boundary that broader adaptive tuning and speed-history heuristics are not yet part of the baseline.

From the scheduling literature, **DRR** is the closest practical borrowing. Its central ideas are byte-based service quanta, deficit carry-over, and suitability for jobs that cannot practically be subdivided during service. That maps unusually well to Pastey's current state, where transfers and current MicroFlowGroup children are still whole file-like work units at scheduler granularity. **WFQ/GPS** should remain the conceptual fairness target, but not the direct implementation: they assume much finer-grained service ordering and virtual-finish-time logic than Pastey can exploit without changing transport semantics. **Plain WRR** is the wrong core model for grouping because it distorts fairness when work units have variable size. **FQ-CoDel's** "new vs old" flow distinction is the best inspiration for a second-stage extension once the dynamic quantum model is stable. [1]

The practical design to ship is a **two-stage model**. In live mode, start with a **stateless dynamic-greedy algorithm**: compute a planner-side `oneWindowQuantumBytes`, derive `dynamicChildCapBytes` and `dynamicGroupCapBytes` from percentages of that quantum, require actual contention before grouping, and keep **one requested window per group** with a **global active micro-group pool of 1** initially. Then, only after shadow data is convincing, add an optional **deficit-mode extension** that gives fairness across multiple waiting micro-flow buckets and later opens a clean path to control/agent lanes. This keeps all semantics in the scheduler, not the protocol. That separation is also consistent with how modern transports such as QUIC and HTTP/3 expose true stream flow control only when the transport is deliberately designed for it, while HTTP prioritization itself has moved toward an extensible, higher-layer scheme rather than rigid transport-coupled semantics. [2]

Current baseline and source priorities

The most important architectural fact in the current repo state is that **Pastey already has the right seam for this change**. The architecture and scheduler docs place pure planning in `src/lib/transferPlanner.ts`, queue/progress readiness and lifecycle management in `src/lib/transferScheduler.ts`, and runtime orchestration plus persistent diagnostics in `src/App.tsx`. The weighted-scheduler design already defines the global planner budget in requested-window terms and already describes the ordinary launch path as batch-relative rather than fixed-class allocation.

The current mismatch is also explicit. The planner policy still exposes fixed MicroFlowGroup limits such as `microGroupMaxChildSizeBytes`, `microGroupMaxGroupBytes`, and `microGroupMaxGroupItems`, and current eligibility additionally depends on file-like status, metadata readiness, active room state, lane, and a grouping key that includes size class and broad MIME family. That means current MicroFlowGroup admission is still threshold-driven even though ordinary requested-window allocation is already weighted and budget-based.

The implementation priorities below follow from that boundary.

Path	Priority	Why it matters for the dynamic algorithm
`src/lib/transferPlanner.ts`	Highest	This should own the new service-quantum model, dynamic child/group caps, contention gating, synthetic group generation, and the fallback `fixed / dynamic_shadow / dynamic` mode switch. It already owns the fixed MicroFlowGroup policy and the weighted window distribution path.
`docs/internal/weighted-scheduler-design.md`	Highest	This should become the normative document for the new “one-window service group” model, because it already defines requested-window budgeting and current weighted allocation semantics.
`docs/internal/phase2-4-scheduling-plan.md`	Highest	This is the right source-of-truth document for boundaries: scheduler-only grouping, serial child dispatch, one-window semantics, and no protocol change. It should carry rollout mode, safety clamps, and when dynamic mode becomes baseline.
`src/lib/transferScheduler.ts`	High	This should stay the source of candidate readiness, queue-state-derived bucket stats, skip reasons, and shadow diagnostics; it should not become the policy owner. This role also fits the recent scheduler-side diagnostic additions you described.

Path	Priority	Why it matters for the dynamic algorithm
`src/App.tsx`	High	This is the reporting and orchestration surface for persistent `[pastey:planner]`, `[pastey:micro-group]`, and `[pastey:runtime-window]` evidence. It is where shadow/live capacity diagnostics should become visible and where rebalance causality should be logged.
`docs/dev/transfer-hot-path.md`	High	This should explain how to validate the new capacity model from a normal `.log` file and how to interpret quantum, cap, skip, and final-summary lines.
`src-tauri/src/commands.rs` / `src-tauri/src/main.rs` / `src/lib/tauri.ts`	Medium	These only matter if the new planner diagnostics add fields or events; they should remain a log bridge, not an algorithm owner.
`CHANGELOG.md`	Medium	This should clearly distinguish “dynamic shadow mode landed” from “dynamic live mode became default.”

The current docs also already define the negative space that should remain untouched: MicroFlowGroup is not a protocol bundle, not archive transfer, not binary-v2, and not transport multiplexing; Phase 4A rebalance is completion-triggered but not yet a general speed-history or throughput-adaptive scheduler. That strongly suggests the next move should be **planner math**, not transport surgery.

What Pastey should borrow from existing schedulers

The first design choice is not “which algorithm is best in the abstract,” but “which algorithm best matches Pastey’s current actuation granularity.” Pastey does **not** currently preempt running file sends, does **not** multiplex multiple logical substreams inside binary-v1, and does **not** schedule at packet or chunk order within a single active transfer. It schedules at the level of **which transfers or groups are launched**, and current MicroFlowGroup children are still dispatched serially. That makes some classic schedulers excellent conceptual guides but poor direct implementations.

The table below summarizes the relevant borrowings.

Algorithm family	What it gives	Why it helps Pastey	Why it should not be copied literally
GPS / PGPS / WFQ	Ideal or near-ideal weighted sharing, work-conserving service, and delay-oriented reasoning. Parekh–Gallager explicitly frame GPS/PGPS as a flexible, work-conserving discipline with throughput and delay guarantees. [3]	It is the right *mental model* for "one window = one share of service" and for keeping long flows from crushing short ones.	Pastey cannot exploit virtual finish-time ordering the way packet schedulers do without finer-grained preemption or transport-level multiplexing. Use GPS/WFQ as the conceptual target, not the direct browser planner implementation.
WRR	Simplicity: integer weights and cyclic service opportunities. Parekh–Gallager's comparison section also shows the basic WRR model. [4]	It explains why a simple "one turn per child" or "count-based" grouping feels intuitive.	WRR distorts service when work units have variable size; packets that miss a slot can wait long enough that the discipline becomes bursty and insensitive to size. For Pastey, files are highly variable, so count-based grouping would be the wrong core metric. [4]
DRR	Byte-based quantum, deficit carry-over, near-fair throughput, and O(1)-style processing. The original DRR paper explicitly highlights applicability to scheduling problems where service cannot be broken up into smaller units. [5]	This is the closest fit. Pastey's current active units are whole transfers or serial child sends, so "credits in bytes/cost" plus carry-over is much more natural than virtual packet finish times.	Full deficit carry-over across all planner cycles adds statefulness and complexity. It is better as a second-stage extension after a stateless dynamic quantum model proves out.
FQ-CoDel's DRR-based flow queueing	Byte credits plus a "new vs old" distinction that favors sparse/latency-sensitive flows while explicitly preventing starvation. RFC 8290 describes this as a byte-based DRR family scheduler with new/old queues and starvation guards. [6]	This is excellent inspiration for future control/agent lanes and for fairness across multiple waiting micro-flow buckets once Pastey has more than one logical low-latency class.	Pastey does not yet need the full state machine. Pull this in only after the core dynamic-capacity model is stable.

Algorithm family	What it gives	Why it helps Pastey	Why it should not be copied literally
Processor-sharing as a user metric target	Dukkipati and McKeown argue that flow completion time is the user-relevant metric and that processor-sharing is a good practical target because it is fair and does not require knowing sizes up front. [7]	This is the best justification for defining a "one-window quantum" at scheduler level: it is a coarse approximation to fair service that still respects user-visible completion time.	Processor-sharing is idealized. Pastey still needs hard caps and contention gates because it cannot deliver continuous sharing with today's serial child path.
pFabric / size-priority transport	Very strong short-flow completion by decoupling scheduling from rate control and pushing fine-grained priorities into the fabric. [8]	It is a useful reminder that *flow scheduling* and *rate control* are distinct problems.	It depends on transport/fabric-level priority semantics. Pastey is intentionally not changing binary-v1 into a true multiplexed transport yet, so this is informative but not directly deployable now. [9]

The practical conclusion is that Pastey should **borrow DRR's quantum-and-credit thinking**, **keep GPS/WFQ as the fairness reference**, **avoid WRR-style count-only grouping**, and **reserve FQ-CoDel's sparse-flow machinery for phase two**. That is also aligned with the transport literature: QUIC and HTTP/3 gain true independent stream progress because the transport itself was designed for flow-controlled streams, while HTTP prioritization has moved toward an extensible, higher-layer signaling model after earlier transport-coupled priority semantics proved too awkward. Pastey should learn from that by keeping "dynamic micro-flow capacity" firmly in the planner unless and until a deliberate transport-v2 exists. [2]

Recommended Pastey algorithm

Design objective

The right new definition is:

> A MicroFlowGroup is not "a set of children smaller than fixed MiB thresholds."
> It is "a set of queued file-like children whose **aggregate service cost** can be reasonably carried by **one planner window share under current contention**."

That definition matters because in Pastey a requested window is **not** a literal transport byte envelope. It is a planner-side concurrency credit. The docs already define the planner in requested-window terms, and MicroFlowGroup is already a scheduler abstraction rather than a transport abstraction. So `oneWindowQuantumBytes` should be treated as an **internal service proxy**, not as "how many bytes physically fit into one transport window."

The second key principle is that grouping should be **contention-sensitive**. If there are only two or three small ready files and the planner has enough active slots and window budget, grouping them serially

wastes available concurrency. Dynamic grouping should therefore be a **resource-pressure response**, not a default path.

Formal model

The recommended first live model is:

```
For each planner-visible file-like item i:

cost_i = remainingBytes_i + perFileOverheadBytes

where:
- remainingBytes_i = sizeBytes_i for queued items
- remainingBytes_i = estimated remaining bytes for active items when known
- perFileOverheadBytes is a planner-side constant, not a protocol field
```

Use `remainingBytes` rather than original size when possible, because the planner is deciding on **current** service pressure, not original batch history.

Then compute a batch-level one-window service quantum:

```
Q = clamp(
      sum(cost_i over planner-visible ordinary file-like work)
      / globalWindowBudget,
      minWindowQuantumBytes,
      maxWindowQuantumBytes
    )
```

Recommended initial semantics:

```
globalWindowBudget            = existing planner budget
minWindowQuantumBytes         = 4 MiB
maxWindowQuantumBytes         = 16 MiB
perFileOverheadBytes          = 256 KiB
dynamicChildCapBytes          = clamp(0.40 * Q, 1 MiB, 4 MiB)
dynamicGroupCapBytes          = clamp(1.00 * Q, 4 MiB, 16 MiB)
minChildrenPerGroup           = 2
maxGroupItems                 = keep existing 32
maxActiveMicroGroupWindows    = 1 initially
```

Those values are deliberately conservative. The fixed thresholds stop being the primary rule, but they still survive as **safety clamps**.

Contention gate

This is the most important new guardrail:

```
effectiveActiveSlotBudget =
  min(safetyActiveTransferCap, globalWindowBudget -
activeFixedWindowReservations)

contentionSeverity =
  runnableOrdinaryFileLikeCount - effectiveActiveSlotBudget

Only attempt dynamic MicroFlowGroup formation when:
- contentionSeverity > 0
- and at least two eligible children exist in a compatible bucket
```

This solves the common pathological case that a pure dynamic threshold model would otherwise create:

- **Two 1.2 MiB files, no contention:** do not group.
- **One huge file plus many 0.3–1.3 MiB files, clear contention:** group.
- **Many tiny files across several buckets, active-slot pressure:** group, but only up to the configured active micro-group pool.

This contention gate is what makes the algorithm genuinely scheduler-aware rather than just a smarter threshold checker.

Cost model and why overhead matters

A pure-byte model undervalues "many tiny items." Dukkipati and McKeown's argument that users care about **flow completion time** more than raw link utilization is directly relevant here: dozens of small items are not free just because their aggregate bytes are small. They create lifecycle work, queue-state churn, progress updates, terminal accounting, and user-visible completion behavior. [7]

That is why the cost model should remain

```
cost = bytes + per-file overhead
```

rather than just `bytes`.

A good starting `perFileOverheadBytes` is **256 KiB**. If shadow mode shows that dynamic grouping is still too reluctant around the 1.1–1.3 MiB regime, slightly reduce overhead; if it over-groups large numbers of ultra-tiny files into long serial groups, increase it. The point of the overhead term is not to simulate protocol bytes; it is to represent **scheduler-visible service cost**.

Grouping key policy

This is where the current design likely needs one deliberate change.

Current eligibility/grouping logic is already constrained by lane, metadata readiness, active room state, file-like status, and a grouping key that includes **size class** and **broad MIME family**. In a fixed-threshold world that is fine, but once dynamic caps define "small enough," keeping **size class** as a hard key starts to reintroduce threshold fragmentation through the back door. A 0.9 MiB file and a 1.2 MiB file may both be dynamically eligible, but using size class as a hard partition can still keep them apart.

The recommended key policy is:

Hard key

- `roomId`
- `lane` (initially still `small_file`)
- `fileLikeKind` or source kind only if it affects dispatch semantics

Soft preference

- `broadMimeFamily`

Do not keep `sizeClass` as a hard grouping key in dynamic mode.

A good live rule is:

1. Bucket primarily by hard key.
2. Prefer grouping within MIME family when enough eligible children exist.
3. If no MIME sub-bucket reaches `minChildrenPerGroup`, allow fallback coalescing across MIME families within the same room/lane bucket.

That gives you grouping locality without needless fragmentation.

Packing strategy

Do **not** turn this into a general-purpose bin-packing optimizer. You do not need that complexity yet, and it will make shadow-vs-live comparisons harder to reason about.

The recommended v1 packer is:

- stable FIFO by queue order,
- within each compatible bucket,
- append eligible children while cumulative `groupCost <= dynamicGroupCapBytes`,
- stop at `maxGroupItems`,
- produce a group only if `children >= 2`.

When several candidate groups are possible but the global active micro-group pool is still 1, rank groups by:

```
savedSlots = children - 1
then packedUtilization = groupCost / dynamicGroupCapBytes
then oldestChildQueueTime
```

That ranking directly serves the real scheduler purpose: reducing active-slot pressure first, then using the one-window budget efficiently, then preserving age fairness.

Optional duration guard

A byte-based quantum model is correct for v1, but a **duration guard** is still a useful secondary safety mechanism once you have enough sender-side history.

Recommended optional guard:

```
estimatedGroupDurationMs =
  groupPayloadBytes / window1ThroughputEwmaBytesPerMs
  + childCount * perChildDispatchPenaltyMs
```

Use it only if you have a stable EWMA for window-1 transfers. If not, disable the guard and log that duration estimation was unavailable.

Suggested initial guard:

- `targetMaxGroupDurationMs = 1000 ms`

If estimated duration exceeds the target, either:

- reduce the pack size until it fits, or
- refuse grouping with `micro_group_skip_reason=duration_guard`.

This is especially valuable on slow links, where even byte-bounded groups can hold one planner window for surprisingly long wall-clock time.

Interaction with existing planner window allocation

This part should remain strictly conservative.

The existing ordinary planner already distributes windows batch-relatively over selected candidates. The new dynamic model should therefore **not** change ordinary requested-window math. Instead, it should inject a synthetic candidate whose semantics are:

```
kind                   = MicroFlowGroup
requestedWindowMin     = 1
requestedWindowMax     = 1
allocationWeight       = groupCost
dispatchMode           = serial_child_dispatch
```

Then let the existing ordinary window allocator continue to do its job for everything else. This preserves the current design principle that a MicroFlowGroup "consumes one planner window" while ordinary large transfers still absorb the majority of the remaining budget.

The initial live policy should also set:

```
maxActiveMicroGroupWindows = 1
```

That preserves the current intuitive huge-plus-micro pattern:

- one active MicroFlowGroup at 1 window,
- large ordinary transfer(s) take the rest,

- completion of the group triggers the normal planner rerun,
- existing completion-only runtime-window rebalance on large transfers remains unchanged.

Optional deficit-mode extension

Only after the dynamic-greedy model is stable should Pastey add a second-stage stateful extension.

The recommended extension is **bucket-level deficit**, not child oversubscription.

Keep the rule that any actual group still satisfies:

```
groupCost <= dynamicGroupCapBytes <= oneWindowQuantumBytes
```

Then add persistent bucket state:

```
bucket.deficitBytes += oneWindowQuantumBytes * bucketWeight
```

A waiting bucket becomes launchable when:

```
bucket.deficitBytes >= nextGroupCost
```

and after launch:

```
bucket.deficitBytes -= nextGroupCost
```

This gives two benefits.

First, if multiple buckets are competing for the single active micro-group pool, they share it fairly over time rather than whichever bucket the greedy chooser happens to like first.

Second, it provides a clean future lane mechanism:

- `small_file` bucket weight = 1
- future `agent_command` or `control` bucket weight > 1
- future `bulk_file` bucket weight = 0 or no deficit grouping

At that point, borrowing **FQ-CoDel's new/old list idea** becomes valuable. Sparse buckets would enter a "new" list and be preferred once, while backlogged buckets cycle through an "old" list with explicit starvation prevention, mirroring RFC 8290's design. [6]

Worked examples

Scenario	Contention?	Quantum result	Expected outcome
Two 1.2 MiB files only	No	`Q` may still clamp high enough to make them individually eligible	**Skip grouping** because there is no active-slot pressure
One 2.7 GiB file + many 0.3–1.3 MiB files	Yes	`Q` will likely hit `maxWindowQuantumBytes`	One dynamic MicroFlowGroup consumes 1 window; huge file remains ordinary and takes most remaining windows
Many 100–900 KiB files, no huge file	Yes once ready-file count exceeds active-slot budget	`Q` likely lands near the lower or middle clamp range	Grouping reduces ordinary runnable count; groups run one-window serially without protocol changes
Many tiny files across different MIME families	Yes	`Q` depends on total visible cost	MIME-preferred grouping first; fallback coalescing across MIME family if no sub-bucket reaches `minChildren`
Slow link with many dynamically eligible children	Yes	`Q` may still allow grouping	Duration guard trims or suppresses oversized groups

Planner integration pseudocode

```
type MicroGroupCapacityMode = "fixed" | "dynamic_shadow" | "dynamic";

interface DynamicMicroGroupPolicy {
  mode: MicroGroupCapacityMode;

  perFileOverheadBytes: number;

  minWindowQuantumBytes: number;   // e.g. 4 MiB
  maxWindowQuantumBytes: number;   // e.g. 16 MiB

  childShareOfWindow: number;      // e.g. 0.40
  groupShareOfWindow: number;      // e.g. 1.00

  minChildCapBytes: number;        // e.g. 1 MiB
  maxChildCapBytes: number;        // e.g. 4 MiB
  minGroupCapBytes: number;        // e.g. 4 MiB
  maxGroupCapBytes: number;        // e.g. 16 MiB
```

```ts
  minChildrenPerGroup: number;     // 2
  maxGroupItems: number;           // keep current 32
  maxActiveMicroGroupWindows: number; // start at 1

  targetMaxGroupDurationMs?: number;
}

interface PlannerVisibleItem {
  queueItemId: string;
  roomId: string;
  lane: "small_file" | "bulk_file";
  fileLikeKind: "file" | "image" | "pasted_image";
  broadMimeFamily?: string;
  queueOrdinal: number;
  status: "queued" | "active";
  metadataReady: boolean;
  activeRoom: boolean;
  cancelled: boolean;
  terminal: boolean;
  bytesRemaining: number;
}

interface DynamicCapacitySnapshot {
  contention: boolean;
  contentionSeverity: number;
  oneWindowQuantumBytes: number;
  dynamicChildCapBytes: number;
  dynamicGroupCapBytes: number;
  ordinaryRunnableCount: number;
  effectiveActiveSlotBudget: number;
}

function computeItemCost(
  item: PlannerVisibleItem,
  policy: DynamicMicroGroupPolicy,
): number {
  return Math.max(0, item.bytesRemaining) + policy.perFileOverheadBytes;
}

function computeDynamicCapacity(
  visible: PlannerVisibleItem[],
  globalWindowBudget: number,
  safetyActiveTransferCap: number,
  activeMicroGroupWindows: number,
  policy: DynamicMicroGroupPolicy,
): DynamicCapacitySnapshot {
  const ordinaryRunnable = visible.filter((x) =>
    x.metadataReady &&
    x.activeRoom &&
    !x.cancelled &&
```

```
    !x.terminal
  );

  const effectiveActiveSlotBudget = Math.max(
    0,
    Math.min(safetyActiveTransferCap, globalWindowBudget -
activeMicroGroupWindows),
  );

  const ordinaryRunnableCount = ordinaryRunnable.length;
  const contentionSeverity = ordinaryRunnableCount -
effectiveActiveSlotBudget;
  const contention = contentionSeverity > 0;

  const totalCost = ordinaryRunnable.reduce(
    (sum, item) => sum + computeItemCost(item, policy),
    0,
  );

  const rawQuantum =
    globalWindowBudget > 0 ? totalCost / globalWindowBudget : totalCost;

  const oneWindowQuantumBytes = clamp(
    Math.round(rawQuantum),
    policy.minWindowQuantumBytes,
    policy.maxWindowQuantumBytes,
  );

  const dynamicChildCapBytes = clamp(
    Math.round(oneWindowQuantumBytes * policy.childShareOfWindow),
    policy.minChildCapBytes,
    policy.maxChildCapBytes,
  );

  const dynamicGroupCapBytes = clamp(
    Math.round(oneWindowQuantumBytes * policy.groupShareOfWindow),
    policy.minGroupCapBytes,
    policy.maxGroupCapBytes,
  );

  return {
    contention,
    contentionSeverity,
    oneWindowQuantumBytes,
    dynamicChildCapBytes,
    dynamicGroupCapBytes,
    ordinaryRunnableCount,
    effectiveActiveSlotBudget,
  };
}
```

```ts
function buildDynamicMicroGroupPlans(
  queuedCandidates: PlannerVisibleItem[],
  snapshot: DynamicCapacitySnapshot,
  policy: DynamicMicroGroupPolicy,
): MicroFlowGroupPlan[] {
  if (!snapshot.contention) return [];

  const eligible = queuedCandidates.filter((item) =>
    item.metadataReady &&
    item.activeRoom &&
    !item.cancelled &&
    !item.terminal &&
    item.lane === "small_file" &&
    computeItemCost(item, policy) <= snapshot.dynamicChildCapBytes
  );

  // Hard key: roomId + lane + fileLikeKind
  // MIME family is treated as a soft preference later, not a hard split.
  const hardBuckets = bucketBy(eligible, (item) =>
    `${item.roomId}|${item.lane}|${item.fileLikeKind}`
  );

  const groups: MicroFlowGroupPlan[] = [];

  for (const bucket of hardBuckets.values()) {
    // Stable FIFO by queue order for determinism.
    const ordered = [...bucket].sort((a, b) => a.queueOrdinal -
b.queueOrdinal);

    let pending: PlannerVisibleItem[] = [];
    let pendingCost = 0;

    for (const item of ordered) {
      const cost = computeItemCost(item, policy);
      const nextCost = pendingCost + cost;

      if (
        pending.length < policy.maxGroupItems &&
        nextCost <= snapshot.dynamicGroupCapBytes
      ) {
        pending.push(item);
        pendingCost = nextCost;
        continue;
      }

      if (pending.length >= policy.minChildrenPerGroup) {
        groups.push(makeMicroGroupPlan(pending, pendingCost, snapshot));
      }

      pending = [item];
      pendingCost = cost;
```

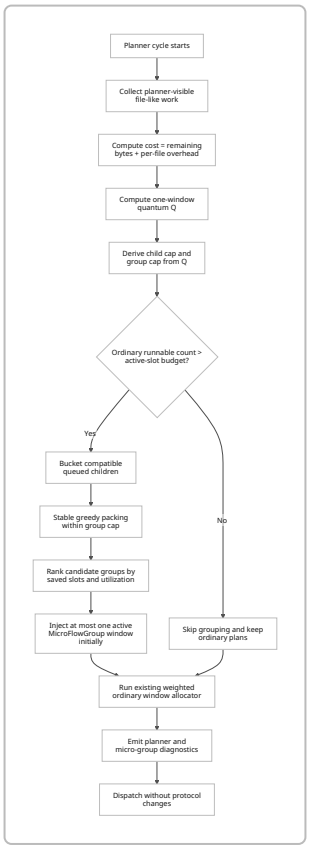
```
    }

    if (pending.length >= policy.minChildrenPerGroup) {
      groups.push(makeMicroGroupPlan(pending, pendingCost, snapshot));
    }
  }

  // Choose only as many as the pool should allow.
  const runnablePool = Math.min(
    policy.maxActiveMicroGroupWindows,
    Math.max(0, snapshot.contentionSeverity),
  );

  return groups
    .sort(compareGroupsBySavedSlotsThenUtilizationThenAge)
    .slice(0, runnablePool);
}
```

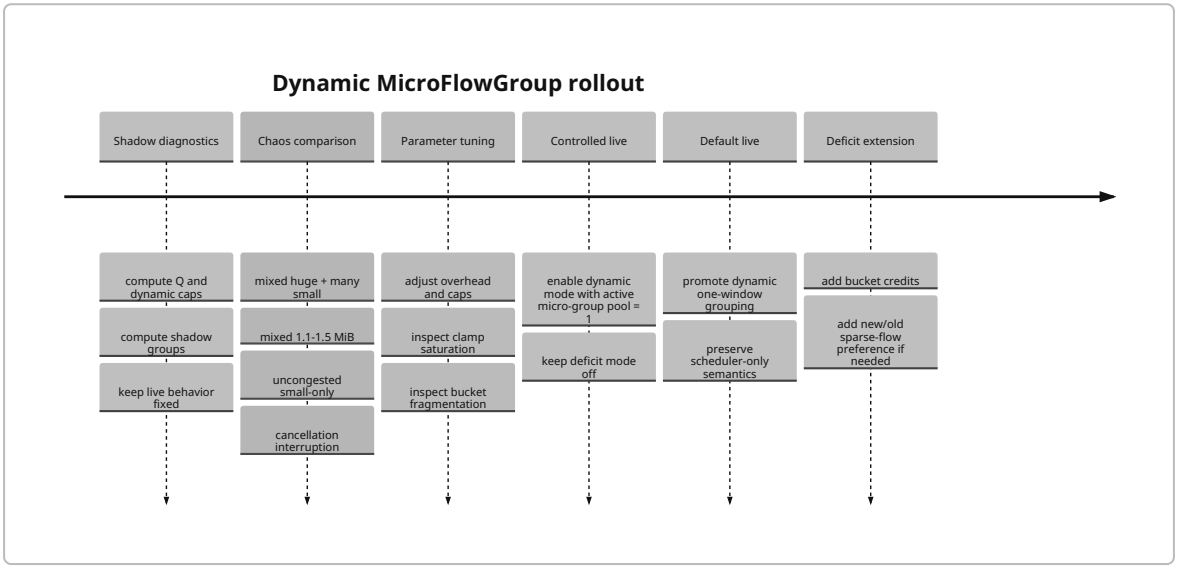
Flow of the recommended planner



Rollout and observability strategy

The right rollout is **shadow first, then constrained live**, because this change is subtle in exactly the way fixed-threshold changes are not. You are changing not only whether groups exist, but *why* they exist.

Recommended rollout phases



The key idea is that shadow mode should tell you **three different things separately**:

- whether dynamic mode would have grouped,
- whether it would have grouped for the *right reason*,
- whether the resulting dynamic caps are usually in-range or constantly slamming into clamps.

Metrics that matter

The table below focuses on metrics that answer the actual strategic question: does the new model improve the scheduler without silently creating longer serial occupancy or larger-file regressions?

Metric	Why it matters	Recommended flip criterion
`one_window_quantum_bytes` distribution	Tells you whether the formula produces a usable dynamic range	Do not promote if it is almost always pinned at min or max
`dynamic_child_cap_bytes` and `dynamic_group_cap_bytes`	Shows whether the cap percentages are sensible	Promote only when mixed batches yield plausible caps for the 1.1–1.5 MiB regime
`dynamic_shadow_micro_group_plans` vs `fixed_micro_group_plans`	Shows whether dynamic mode actually fixes the current gap	Promote only if it captures the mixed-small workloads you currently miss

Metric	Why it matters	Recommended flip criterion
`micro_group_skip_reason=no_contention` rate	Confirms the algorithm is not serializing uncongested batches	High in low-fanout batches is good
`bucket_fragmentation` indicators	Shows whether hard grouping keys are still too narrow	If fragmentation remains high, soften MIME or remove more hard keying before promotion
`packed_cost / dynamic_group_cap`	Shows whether packer utilization is sensible	Promote only when most accepted groups are neither extremely underfilled nor constantly right at the edge
Small-file completion time	This is the user-visible target	Promote only if p50/p95 small-file completion improves or is at least not meaningfully worse
Large-file completion regression	Ensures the 7+1 style behavior is preserved	Promote only if large-file p95 completion does not regress materially
`runtime-window` summaries on large transfers	Confirms completion-triggered rebalance still behaves as expected	Large transfers should still gain freed windows after group terminal events
Group `cancelled / interrupted / completed_with_errors` ratio	Ensures no lifecycle regression	Promote only if these rates stay flat

Log fields to emit

The current persistent diagnostic path already makes this rollout feasible. The following fields are the minimum high-value additions for dynamic shadow/live mode.

For `[pastey:planner]`:

```
event=launch_summary
micro_group_capacity_mode=fixed|dynamic_shadow|dynamic
```

```
ordinary_runnable_count=...
effective_active_slot_budget=...
contention=true|false
contention_severity=...
per_file_overhead_bytes=...
one_window_quantum_bytes=...
dynamic_child_cap_bytes=...
dynamic_group_cap_bytes=...
eligible_children_fixed=...
eligible_children_dynamic=...
fixed_micro_group_plans=...
dynamic_shadow_micro_group_plans=...
dynamic_shadow_grouped_children=...
largest_dynamic_bucket_children=...
largest_dynamic_bucket_cost_bytes=...
micro_group_window_pool=...
micro_group_skip_reason=no_contention|insufficient_children|
child_cost_exceeds_cap|bucket_fragmentation|duration_guard
```

For `[pastey:micro-group]`:

```
event=planned
group_cost_bytes=...
payload_bytes=...
dynamic_group_cap_bytes=...
children=...
saved_slots=...
grouping_key=...
mime_policy=preferred|coalesced
estimated_duration_ms=...
duration_guard_applied=true|false
requested_window=1
```

For `[pastey:runtime-window]`:

```
event=summary
room_id=...
queue_item_id=...
transfer_id=...
initial_window=...
final_window=...
min_window=...
max_window=...
update_count=...
terminal_status=completed|failed|cancelled|interrupted
rebalance_trigger=ordinary_terminal|micro_group_terminal|manual_cancel
```

The most important new causal field is `rebalance_trigger=micro_group_terminal`, because it connects group completion back to the Phase 4A window-mutation story already in the baseline.

Example log lines

```
[pastey:planner] event=launch_summary room_id=7c...
micro_group_capacity_mode=dynamic_shadow ordinary_runnable_count=11
effective_active_slot_budget=4 contention=true contention_severity=7
per_file_overhead_bytes=262144 one_window_quantum_bytes=16777216
dynamic_child_cap_bytes=4194304 dynamic_group_cap_bytes=16777216
eligible_children_fixed=1 eligible_children_dynamic=9
fixed_micro_group_plans=0 dynamic_shadow_micro_group_plans=1
dynamic_shadow_grouped_children=8 largest_dynamic_bucket_children=10
largest_dynamic_bucket_cost_bytes=14548992 micro_group_window_pool=1

[pastey:micro-group] event=planned room_id=7c... group_id=mg_01
requested_window=1 children=8 saved_slots=7 payload_bytes=12341248
group_cost_bytes=14438400 dynamic_group_cap_bytes=16777216 grouping_key=room:
7c...|lane:small_file|kind:file mime_policy=coalesced
estimated_duration_ms=820 duration_guard_applied=false

[pastey:runtime-window] event=summary room_id=7c... queue_item_id=q_09
transfer_id=t_55 initial_window=7 final_window=8 min_window=7 max_window=8
update_count=1 terminal_status=completed
rebalance_trigger=micro_group_terminal
```

Grep commands

```
grep -E "\[pastey:planner\].*micro_group_capacity_mode=" pastey*.log

grep -E "one_window_quantum_bytes=|dynamic_child_cap_bytes=|
dynamic_group_cap_bytes=" pastey*.log

grep -E "\[pastey:planner\].*micro_group_skip_reason=" pastey*.log

grep -E "\[pastey:micro-group\].*event=(planned|final)" pastey*.log

grep -E "\[pastey:runtime-window\].*event=summary" pastey*.log
```

Risks and migration checklist

The central risks are not difficult to name; the dynamic model becomes robust only when each risk is paired with a concrete guardrail.

Risk	Why it appears	Recommended mitigation
Over-grouping in uncongested batches	Dynamic caps alone can make 1.1–1.5 MiB files “eligible” even when there is no reason to serialize them	Require **contentionSeverity > 0** before any dynamic group is live
Quantum inflation from huge files	`sum(cost)/globalWindowBudget` can become very large when one or two huge transfers dominate	Keep hard `Qmax`, start conservative, and only add winsorization if shadow logs show chronic max-clamp saturation
Grouping-key fragmentation	Hard `sizeClass` or overly rigid MIME partitions can keep dynamically eligible files apart	Remove `sizeClass` from hard key; make MIME a soft preference rather than an absolute partition
Long serial occupancy on slow links	A group that is byte-bounded may still be time-expensive on weak links	Add optional `targetMaxGroupDurationMs` using window-1 EWMA throughput when available
Too many active groups starving bulk	Even one-window groups can eat too many planner opportunities if launched freely	Start with `maxActiveMicroGroupWindows = 1`; only raise later with evidence
Deficit-mode complexity too early	Persisting per-bucket state adds subtle fairness and lifecycle edge cases	Keep live v1 stateless; defer deficit mode to phase two
Confusing observability	Without explicit quantum/cap/skip logs, shadow data will be uninterpretable	Make planner logs carry quantum, caps, contention, bucket size, and skip reason explicitly
Accidental transport creep	Dynamic grouping can tempt protocol coupling	Keep all semantics in `transferPlanner.ts` and scheduler logs unless a deliberate transport-v2 project is started

The migration checklist should therefore stay short and disciplined.

First, update the normative docs:

- `docs/internal/weighted-scheduler-design.md` should define the service-quantum model, formulas, clamps, contention gate, and hard/soft grouping key rules.
- `docs/internal/phase2-4-scheduling-plan.md` should record the shadow/live rollout stages and the fact that dynamic grouping remains scheduler-only and one-window-only.
- `docs/dev/transfer-hot-path.md` should explain how to inspect quantum, cap, skip, and runtime-window summary lines from the persistent `.log`.
- `CHANGELOG.md` should clearly distinguish “dynamic shadow mode landed” from “dynamic mode enabled by default.”

Second, keep the implementation boundary clean:

- `src/lib/transferPlanner.ts` owns formulas and group synthesis.

- `src/lib/transferScheduler.ts` owns readiness stats and shadow counters.
- `src/App.tsx` owns persistent reporting.
- `commands.rs`, `main.rs`, and `src/lib/tauri.ts` should only be touched if new log fields need the existing bridge surface.

Third, keep the existing CI/build sanity pass unchanged even if you do not add fresh code-level tests in this phase:

```
rtk npm run build
rtk node scripts/run-transfer-planner-tests.mjs
rtk cargo test frontend_diagnostic_log
rtk git diff --check
rtk npm run build:checked
```

The final design decision is therefore fairly crisp:

Pastey should move from “tiny file thresholds” to “one-window service groups,” but only under contention, only with conservative clamps, only with one requested window per live group at first, and only as a scheduler policy—not as a transport feature.

That path is the best fit for the current architecture, the cleanest continuation of the existing weighted planner, and the safest bridge toward future control or agent-command lanes.

---

[1] [5] https://courses.cs.duke.edu/fall24/compsci514/readings/drr.pdf
https://courses.cs.duke.edu/fall24/compsci514/readings/drr.pdf

[2] https://datatracker.ietf.org/doc/html/rfc9000
https://datatracker.ietf.org/doc/html/rfc9000

[3] [4] https://pbg.cs.illinois.edu/courses/cs598fa09/readings/pg93.pdf
https://pbg.cs.illinois.edu/courses/cs598fa09/readings/pg93.pdf

[6] https://datatracker.ietf.org/doc/rfc8290/
https://datatracker.ietf.org/doc/rfc8290/

[7] https://people.eecs.berkeley.edu/~sylvia/cs268-2019/papers/rcp-ccr.pdf
https://people.eecs.berkeley.edu/~sylvia/cs268-2019/papers/rcp-ccr.pdf

[8] [9] https://web.stanford.edu/~skatti/pubs/sigcomm13-pfabric.pdf
https://web.stanford.edu/~skatti/pubs/sigcomm13-pfabric.pdf